

Proposal: Sandboxed PVM Worker Boundary

Decision proposed: move all untrusted actor execution out of the validator and into a sandboxed, single-threaded worker that forks a clean interpreter snapshot for every actor frame.

2026-07-31 — revised with on-chain validation on a live validator.

Executive summary

This document proposes one durable fix: **the validator should never execute actor-supplied Python in its own process.** Instead, a long-lived, single-threaded PVM worker establishes its sandbox at boot, warms one deterministic RustPython interpreter, and forks a child from that clean snapshot for every actor frame. The child closes inherited file descriptors, tightens its seccomp profile, runs exactly one actor frame, returns a bounded canonical result, and exits. The validator remains the authority for chain state and accepts only typed, validated effects over a private IPC channel.

That boundary supersedes the finding-by-finding remediation strategy. Once every actor frame runs only in the constrained child, `std::process::exit`, `SIGKILL`, an interpreter panic, a native module, or a mutated Python registry can terminate or affect only that child; the validator and the pristine zygote remain intact. Fork-per-frame also removes interpreter reuse across actors by construction, so correctness no longer depends on maintaining a complete reset/freeze inventory across Python objects, VM fields, Rust statics, thread-locals, native modules, and future RustPython upgrades.

The proposed boundary is more than a `fork()` call. Four properties are part of the fix and must ship with it:

1. **Per-actor-frame isolation from a pristine zygote**, including nested `call_actor` frames.
2. **A capability-minimal child**, narrowing today's import allowlist (§7.2) to a capability-minimal one and extending positive construction to the parts of the environment that have none, deterministic namespace values, explicit fd closure, Landlock, and a child seccomp profile that denies process creation, networking, and host filesystem access.
3. **A trusted broker contract**, where state reads and typed effects are actor/transaction-bound, canonical, size/gas bounded, and committed through a staged overlay only after parent-side validation.
4. **Deterministic in-loop metering and termination**, so every validator stops actor code at the same instruction/gas boundary; wall-clock timeout, OOM, signal, malformed IPC, and unexpected child death are fail-stop infrastructure events, never committed transaction outcomes.

The spike measured about **1.2 ms of marginal fork/CoW/teardown cost** versus **86-133 ms** for building a fresh interpreter. Those are scratch aarch64-in-Docker measurements, not production capacity numbers, but they establish fork as the best-performing isolation design measured so far. The expensive part of the epic is not fork; it is extracting the current synchronous state reads and at least ten direct store-write paths into the brokered, staged transaction contract.

Decision requested. Approve the sandboxed-worker boundary as the target architecture; treat per-frame isolation, deterministic metering, and the parent-side effect/read contract as acceptance criteria rather than follow-ups; and sequence implementation around the contract first. The existing live issues in Section 7 remain useful as regressions and may receive small bridge fixes while the epic is built, but they should no longer define the remediation strategy.

Scope and confidence. This proposal is based on source inspection at `cowboyinc/node origin/devnet cdb89a4e`, a production-config reproduction of the checkpoint exit (now also confirmed on-chain), an uncommitted production-path reproduction of `settrace`, on-chain confirmation of the `posix/io` escapes on a live validator, and scratch isolation/performance spikes. Validate the design with committed production-path regressions, a live-box transaction profile, and bare-metal measurements before locking capacity or rollout parameters. The `posix` root command execution, the checkpoint validator halt, and the `io-wrapper` host file read/write are **confirmed live on-chain** (river, node `v0.0.34-0efc59f` = devnet HEAD; register §6.3). Circulate this proposal only within the engineering/security need-to-know group until they are contained.

1. The proposed boundary

1.1 Architecture at a glance

```
trusted validator / transaction broker
- owns chain state and transaction overlay
- validates reads, calls, effects, quotas, gas, and result encoding
- never imports or executes actor code
    |
    | private, versioned, bounded IPC
    v
single-threaded PVM zygote worker
- sandboxed at boot: namespaces, pivot_root, Landlock, baseline seccomp
- deterministic warm-up and capability-minimal actor environment
- never handles actor work itself; it only forks pristine snapshots
    |
    | fork one clean child per actor frame
    v
execution child
- close-range / fd allowlist
- tighter seccomp: no fork/clone/exec, network, or ambient host access
- execute one frame under deterministic instruction/gas budget
- request bounded reads/calls; return typed effects + result; _exit
```

The trust boundary is the process boundary between the validator/broker and the worker/child. RustPython remains a rich interpreter, but it is no longer asked to sandbox itself. The child is treated as compromised from the first actor instruction: it has no chain-state authority, no validator-facing socket, no storage or log descriptors, and no ability to commit its own output.

1.2 Why the zygote is long-lived and single-threaded

A cold RustPython build measured 86-133 ms. A warmed zygote makes the same logical interpreter image available through `fork()` for about 1.2 ms of marginal overhead. The zygote must be provably single-threaded and created before tokio or any thread pool: after a multi-threaded fork, a lock held by a vanished thread can remain locked forever in the child. The current `block_in_place / Handle::block_on` bridge in `pvm_host.rs:328-345` therefore cannot live inside the forking process.

The zygote does not execute actors and does not become dirty. Every actor child is disposable; it never re-enters the zygote and never becomes the parent of a later actor. This is the property that replaces the warm-interpreter pool's growing reset/freeze inventory.

1.3 Isolation granularity: one child per actor frame

The security invariant is:

Execution of actor B must not depend on mutable interpreter state left by actor A, including when A and B are siblings in one transaction call tree.

Fork-per-transaction does not satisfy that invariant if nested actors reuse one child or a pool inside the transaction. `call_actor`, `fork_actor`, synchronous event delivery, and token hooks create nested actor frames; a transaction-level child would still allow one actor to affect a later sibling.

The proposed granularity is therefore **one zygote child per actor frame**. A nested call is broker-mediated:

1. The caller child sends a typed `CallActor` request and suspends.
2. The trusted broker validates the call against the current transaction overlay.
3. The zygote forks a new clean child for the callee.
4. The callee result and typed effects are validated and merged into the overlay.
5. The broker returns a canonical result to the suspended caller.

This is more integration work than one child per transaction, but it is the granularity that actually delivers “no interpreter reuse across actors.”

1.4 The properties this boundary buys

Property	Mechanism	Result
Validator liveness isolation	Actor code runs only in a disposable child	<code>exit</code> , <code>SIGKILL</code> , and a child panic do not terminate the validator
No cross-actor interpreter state	Every frame forks from the unchanged zygote	No warm state passes from one actor to another
Host capability containment	Mount/UTS/ user namespaces, <code>pivot_root</code> , Landlock, fd closure, child <code>seccomp</code>	Native modules cannot reach the validator host or spawn an unsandboxed process
Least authority	Default-deny actor globals/modules and no privileged descriptors	An interpreter escape still lands in a capa- bility-minimal process
State integrity	Parent owns reads, overlay, validation, and commit	A compromised child can propose effects but cannot authorize them
Deterministic termination	In-loop instruction/gas budget	All validators stop the same actor at the same logical point
Upgrade resilience	Private memory is discarded with the child	RustPython upgrades do not reopen a per-field reset checklist

2. The validator-child contract

The process split is only correct if the child is untrusted and the validator remains authoritative. “The child returns a write set” is insufficient by itself: a memory-safety escape inside the child could fabricate any payload the protocol accepts.

2.1 Request identity and canonical framing

Every message must be bound to:

- protocol version;
- transaction hash and execution-frame id;
- requested actor and caller;
- chain/epoch context;
- monotonically increasing request sequence;
- byte, item-count, and gas limits.

Encoding must be canonical: one logical value has one byte representation; unknown fields, duplicates, non-canonical ordering, oversized lengths, and trailing bytes are rejected. The deserializer belongs to the trusted boundary and must be fuzzed as panic-free.

2.2 Reads: a transaction-consistent overlay

Actor execution currently performs synchronous, data-dependent reads in the middle of execution (`state_get`, `get_actor_for_call`, `load_callee_pins`, `switch_to_callee`, scans). The key set depends on actor control flow, so a complete pre-fetched read set is not generally available.

The broker therefore needs a bounded read-back protocol:

- authorize every key/range against the current actor and operation;
- serve reads from the transaction overlay first, then the pinned base snapshot;
- attach read versions for conflict validation;
- meter request count, returned bytes, and scan work;
- return canonical values only;
- never pass a storage handle or validator fd into the child.

This gives every child read-your-writes semantics while preserving one transaction view across nested frames.

2.3 Writes and side effects: typed proposals, parent authorization

At least ten current host paths write the store during execution and bypass `rollback()`: the CIP-29 subscription mutators, `acquire_epoch_access`, `register_account_key`, the SKM branch of `call_actor`, synchronous `emit_event`, and `econ_intent_gate`. `ExecutionSideEffects` currently covers only messages, timers, and events; storage cache writes, token queues, pending maps, balance deltas, code bytes, actor nonce, and entitlement counters sit elsewhere.

The target contract is a single mechanically enforced choke point:

1. The child emits typed effects, never raw store writes.
2. The broker re-validates actor/caller capability, namespace, quotas, amounts, nonces, gas, and effect ordering.
3. Effects enter the transaction overlay and are visible to later reads in the same transaction.

4. Commit occurs once, after the full execution tree succeeds.
5. Any actor failure discards the overlay; malformed child output is an infrastructure fault.

Typed effects should replace hand-encoded byte queues such as `token_id || to || amount`. The parent, not the child, assigns authoritative ids or values that depend on live state.

2.4 Nested calls and suspension

The broker must support a suspended caller while it runs a clean callee frame. A child may receive only the result of an authorized nested operation, not direct access to the callee, store, scheduler, or another child endpoint. Depth, fan-out, request bytes, and aggregate gas are transaction-scoped limits.

The private endpoint is one-shot and frame-specific. EOF, wrong sequence, duplicate completion, or a response for another frame is rejected. A child never inherits the validator RPC listener, storage descriptors, logs, or another child’s channel.

3. Deterministic execution and termination

Process isolation contains a failure; it does not by itself make the decision to terminate deterministic. A validator that commits “timeout” while another finishes the actor would fork the chain.

3.1 Required stop semantics

Stop trigger	Classification	Protocol action
Instruction/gas budget exhausted at a deterministic interpreter point	deterministic	Commit the canonical “transaction failed; effects discarded” outcome
Defined deterministic PVM exception/trap	deterministic	Commit the canonical failure outcome
Unexpected Rust panic, signal, OOM, wall-clock watchdog, seccomp kill, malformed/EOF IPC	infrastructure-dependent	Fail-stop/replay this validator; never commit it as a transaction result

Unexpected child death may be isolated from the validator process, but it is not automatically a deterministic transaction outcome. The supervisor records the diagnostic and causes the validator to drop out or replay according to consensus policy.

3.2 In-loop metering is part of the boundary

COW-2419 records per-instruction work into `cycles_used`, but `pvm_instr_gas_price_ppm` defaults to 0, and `frame.rs:455-467` only accumulates cost; there is no in-loop limit check. The 2026-07-14 mesa halt showed why fee computation after execution is insufficient: two 500-900 ms timer handlers pushed proposal time beyond the 1000 ms leader timeout while gas remained approximately zero.

The child must check a transaction-scoped instruction/gas budget inside the interpreter loop and stop at a deterministic point. A wall-clock watchdog remains useful for local containment, but its firing is an infrastructure fault, not a committed result.

3.3 Deterministic environment

Fork gives each validator’s child the same logical zygote state, not byte-identical virtual addresses across machines. The actor environment must therefore continue to pin or remove observable box inputs:

- deterministic object identity/repr behavior;
- no raw thread ids;
- deterministic/hidden hostname and process arguments;
- deterministic time/randomness;
- no ambient filesystem/network results;
- canonical errors independent of kernel wording.

A narrowed module allowlist plus a positively constructed global namespace is part of the child profile — the module half tightens a list that already exists (§7.2); the global half is new. Process isolation contains `_socket.gethostname` or `_thread.get_ident`; it does not make their return values deterministic if they remain actor-visible.

4. Sandbox lifecycle and syscall policy

4.1 Worker boot

Before any actor execution, the worker:

1. enters dedicated user, mount, PID, network, IPC, and UTS namespaces as supported;
2. mounts a minimal read-only runtime, applies `pivot_root`, and removes the old root;
3. sets deterministic namespace-visible values such as hostname;
4. applies Landlock to the minimal runtime tree;
5. closes every non-protocol descriptor;
6. initializes and deterministically warms RustPython;

7. proves no additional thread exists;
8. installs a baseline seccomp profile that permits only zygote operation, IPC, `fork/clone` as narrowly required, `wait4`, memory management, and exit, then enters the fork service loop.

`0_CLOEXEC` is not sufficient because the design forks without `exec`; descriptor closure or an explicit allowlist is required.

4.2 Child setup before actor code

Immediately after fork, the child:

1. keeps only its private broker endpoint;
2. inherits per-frame rlimits and is placed into its cgroup by the trusted supervisor before actor execution;
3. tightens seccomp to deny `fork/clone`, `execve`, sockets, cross-process signals, ptrace, namespace changes, and filesystem mutation outside the minimal runtime;
4. installs the frame budget and deterministic execution context;
5. runs one actor frame;
6. writes one bounded completion record and calls `_exit`.

The tightened child profile must be installed before any actor import or bytecode. Actor-visible `_thread` creation is incompatible with the target child profile and should not be present in the constructed environment.

4.3 What remains trusted

The validator/broker, IPC parser, effect validator, overlay, commit path, and zygote bootstrap remain trusted. Fork isolates faults in actor execution; it does not protect the validator from a panic in parent-side deserialization, effect validation, or commit. Those paths require fuzzing, bounded allocation, and an unwind-safe all-effects-staged commit boundary.

5. Feasibility and cost

5.1 Measured marginal cost

Scratch spike: `examples/zygote_spike.rs`, aarch64 Linux in Docker. Warmed interpreter: 24.2 MB. Observed actor write set: about 2.6 MB / 653 pages.

Component	Measured value
<code>fork()</code> return, serialized on execution thread	63-89 us
CoW page faults, about 653 x 1.25 us	about 816 us
teardown, <code>_exit</code> / <code>waitpid</code> / pipe	about 300 us
marginal isolation cost	about 1.18-1.20 ms; measured p50 range 0.95-1.32 ms
in-process warm reference	0.28-0.33 ms
cold interpreter build	86-133 ms

The spike supports fork as the practical isolation mechanism among the options measured. It does not establish production throughput, worker count, pool depth, p99 latency, or bare-metal cost.

5.2 Capacity is driven by handler time, not fork time

Worker occupancy is `fork overhead + handler runtime + broker waits`. The mesa evidence includes handlers at 500-900 ms, so handler runtime dominates the 1.2 ms marginal boundary cost. Capacity planning must use **actor-frame executions**, not transaction count, because one transaction can fan out through nested calls, hooks, timers, and synchronous events.

Measure the live distribution of:

- actor frames per block and per transaction;
- handler wall time and deterministic instruction count;
- nested depth/fan-out;
- read-back count and bytes;
- typed effect count and bytes;
- p50/p95/p99 child lifetime and worker queue time.

5.3 Optional pre-faulting is not part of the first release

An experimental child can privatize writable mappings before receiving work, moving CoW faults off the hot path. The spike reduced execution-time CoW to zero and put pure compute near 267 us, but background pre-faulting cost about 4.8 ms and roughly 14 MB resident per ready child. At current low load, build a pool abstraction but do not enable eager pre-faulting. Revisit it only when measured throughput justifies the memory and copy amplification.

6. Implementation and rollout

6.1 Slice 1 - extract the trusted transaction contract

- Inventory every state read and store mutation reachable during actor execution.
- Introduce a read-through transaction overlay with read versions.
- Route every mutation through typed, parent-validated effects.
- Establish one commit/rollback choke point and mechanically prevent direct writes from the PVM host.
- Fuzz the canonical request/result/effect parser and enforce byte/count/gas bounds.

This is the opening slice because the worker cannot be safely separated until the validator can serve reads and commit effects without giving the child authority.

6.2 Slice 2 - build the zygote and disposable child runner

- Start a single-threaded worker before any async runtime.
- Apply the worker namespace, root, Landlock, fd, and baseline-seccomp setup.
- Construct the positive actor builtins/globals/module set and pin deterministic environment values.
- Warm a deterministic interpreter and fork one child for a minimal actor frame.
- Apply the tighter child profile before actor code.
- Return a canonical result through a one-shot endpoint.
- Verify `exit`, `SIGKILL`, `panic`, `fork/exec`, `network`, and `host-file` probes affect only the child and cannot reach host capabilities.

6.3 Slice 3 - broker reads, effects, and nested frames

- Move synchronous host reads behind the bounded broker protocol.
- Run every nested actor frame in a fresh zygote child.
- Preserve transaction-overlay read-your-writes behavior across the execution tree.
- Enforce depth, fan-out, total request bytes, effects, and aggregate gas.
- Remove direct access to `tokio`, `storage`, `scheduler`, and `privileged descriptors` from the worker.

6.4 Slice 4 - deterministic metering and failure semantics

- Enforce the instruction/gas budget inside the interpreter loop.
- Define canonical protocol errors for budget exhaustion and PVM traps.
- Treat unexpected `panic`/signal/OOM/IPC failure as validator fail-stop/replay.
- Add an unwind-safe staged boundary around parent validation and commit.

6.5 Slice 5 - differential rollout

- Run the existing engine and worker engine in shadow/differential mode over a committed corpus.
- Compare result, gas, reads, typed effects, and final overlay.
- Add one channel-specific regression for every finding in Section 7 / Appendix A.
- Exercise production `execute_handler`, nested calls, token hooks, timers, and synchronous events.
- Roll out on devnet with telemetry before making the worker path consensus-authoritative.
- Retire the warm interpreter pool after equivalence and isolation gates pass.

6.6 Exit criteria

The epic is complete only when:

- no validator process imports or executes actor code;
- every actor frame, including nested frames, starts from the unchanged zygote;
- the child inherits only one private endpoint and cannot create threads/processes or access ambient host capabilities;
- every read/effect is bounded, canonical, actor/transaction-bound, and parent-authorized;
- all transaction effects are staged and committed once;
- deterministic budget exhaustion is enforced in-loop;
- unexpected child death can never become a committed transaction outcome;
- the production-path differential/isolation corpus is green.

7. Existing issues this boundary supersedes

The findings below are evidence for the boundary, not the proposed work breakdown. Small bridge fixes remain prudent until the worker path is live, but the target design should make the class unreachable or child-local rather than maintaining one patch per channel.

7.1 Actor-triggerable validator halt (and a reachable-but-non-firing signal path)

`rustpython_checkpoint` reaches `std::process::exit(0)` (COW-2731)

An actor can import the whitelisted module and call `checkpoint_bytes()`. The request is checked every instruction and reaches `std::process::exit(0)` (`frame.rs:516,595,2789-2807`). Production uses `continuation: None`, leaving `checkpoint_exit=true`; a default-options harness masks the issue because continuation mode forces it false. This was reproduced with production-config options, and **confirmed live on-chain** (river, node `v0.0.34-0efc59f` = devnet HEAD): a deployed actor halted the validator, `RestartCount` 0→1. COW-2731 is In Progress; PR #1110 is not merged to devnet, so the kill fires at HEAD.

Under the proposed boundary: the module is absent from the positive actor environment, so the call is unreachable. If an equivalent interpreter route reappears, process isolation limits it to the disposable child; the validator never executes the exit.

signal.raise_signal(SIGKILL) — reachable, but did not kill the validator on-chain
`_signal` is registered, exports `SIGKILL`, and calls `libc::raise` (`signal.rs:119,449,470`); it is reachable from actor code via a nested import (an earlier draft treated this as a validator kill on that basis). **On-chain, it did not fire:** a deployed actor that called `_signal.raise_signal(_signal.SIGKILL)` survived the call, validator `RestartCount` unchanged (register §2.2). It is kept here as a reachable native capability, not a demonstrated kill; why the `SIGKILL` does not terminate the process is an open question.

Under the proposed boundary: `_signal` is absent from the positive actor-module set and the child profile denies cross-process signal syscalls regardless — a reachable signal call could affect only the disposable child.

7.2 The actor environment is allowlisted at the import boundary and unconstrained inside it

COW-2434’s freeze/write barrier is effective for the object mutations it covers. The residual issue is coverage, and it is worth stating precisely because the obvious version of this claim is wrong.

Module imports are **already** default-deny against a positive allowlist: `_is_allowed()` checks the deny list, then the allow list, then the per-call library list, then prefix matches, and falls through to `return False` (`guard.rs:68-97`); `_pvm_import` raises for anything that fails it (`guard.rs:149-161`). A positively constructed module set is therefore not missing at the import boundary.

The gap is in three other places. **What the allowlist admits:** `_signal`, `_socket`, `_posixsubprocess`, `_ssl`, `fcntl`, `sys`, `io`, `math`, and `_thread` are all on it, so §7.1’s checkpoint halt and the reachable `_signal` path are both permitted imports. That part is bounded work on a list we control. **When the allowlist is consulted at all:** `import_inner` (`vm/mod.rs:821`) runs the Rust deny check (`vm/mod.rs:654-675`) and then returns a `sys.modules` hit directly (`vm/mod.rs:851-860`); only a cache miss reaches `builtins._import` and therefore the allowlist. Any module resident in `sys.modules` is thus reachable without the allowlist being consulted — see the `_io` route in §7.5. **What remains reachable inside an admitted module:** nothing constrains the attribute surface. Once `sys` is importable every attribute on it is reachable; once `io` is importable — and it must be — `FileIO` arrives with it, which is why the guard could correctly replace `open` and still leave host file I/O reachable beside it. The admitted-module set is enumerable; the attribute surface hanging off it is not, and it grows with each RustPython release rather than with anything we write.

That is the durable argument for moving the boundary: shrinking the allowlist is finite and worth doing, but no name-based control reaches the second problem. The uncovered surface spans five layers:

Layer	Representative channel	Consequence today	Boundary property
VM fields	<code>sys.settrace</code> / <code>setprofile</code> retained in public VM fields	code runs in a later actor	new child per frame
Rust/Python registries and TLS	<code>asyncgen</code> hooks, <code>codecs</code> , <code>atexit</code> , closure cells	later execution observes/mis- uses state	child private memory is dis- carded
<code>_thread</code>	spawned thread bypasses thread-local barrier; raw thread id	cross-actor mutation / consen- sus input	no actor threads; child sec- comp + minimal environment
Native modules	<code>_signal</code> , <code>_posixsubprocess</code> , <code>_socket</code> , <code>_ssl</code> , <code>fcntl</code>	process kill, fork/exec, net- work, box inputs	narrower allowlist + kernel sandbox
Ambient environment	<code>io.FileIO</code> , <code>sys.orig_argv</code> , hostname	host I/O / validator-local out- put	pivoted root, Landlock, de- terministic namespace, pinned globals

More than twenty identified channels are listed in Appendix A. Eight representative primitives are reachable on `devnet` today; `_ssl` and `fcntl` remain unassessed members of the same native class and are not counted as confirmed live. (The allowlist also contains the string `_fcntl`, but no module of that name is registered — the Unix module is `fcntl` — so that entry grants nothing.)

7.3 Cross-actor state

`sys.settrace(f)` stores `f` in `vm.trace_func`; a later pooled frame fires it from `callable.rs:51,53`. The pool key carries no actor address or code hash. The mechanism is source-verified and was reproduced through production `PvmExecutor::execute_handler` using an uncommitted harness; the committed #1105 corpus uses a lower-level runtime path.

`_thread.start_new_thread` creates a second RustPython thread whose `BARRIER_ACTIVE` thread-local defaults false (`sandbox.rs:27,107-118`). That thread shares the interpreter baseline but never enters `BarrierScope`, so it can mutate objects the main actor thread cannot.

Under the proposed boundary: no child survives a frame, no actor can create a thread, and every sibling actor starts from the unchanged zygote.

7.4 Consensus inputs

Known actor-visible box inputs include:

- cold/warm `id()` sequence divergence;
- `_thread.get_ident`;
- `sys.orig_argv`;
- `_socket.gethostname`;
- mutable process/TLS configuration such as socket default timeout and integer-string limits.

Fork-per-frame removes cold/warm history, but process isolation alone does not make host values deterministic. That is why the minimal actor environment and deterministic UTS/globals are explicit acceptance criteria in Sections 1 and 3.

7.5 Host escape and validator liveness

`_posixsubprocess.fork_exec` reaches real `fork()` / `execve`; `_socket` creates real kernel sockets; `io.FileIO` reaches `crt_fd::open` even though `builtins.open` is blocked. An uncaught actor-execution panic currently propagates through a validator configured with `.with_catch_panics(false)`, terminating consensus.

The sharpest instance is **arbitrary command execution as root, confirmed on-chain**. A deployed actor running `def main(...): import posix; posix.system(...)` ran `id` (returned `uid=0(root)`) and read `/etc/shadow` on a live validator (register §2.3). It composes the §7.2 gaps: `posix` is `os` under its Unix name, preloaded and on no list; a *nested* import evades the top-level deploy resolver while the full-AST deny-scan does not list `posix`; and the `import` statement returns from the `sys.modules` cache before the allowlist is consulted. Host file read/write is confirmed the same way through the allowlisted `io` wrapper (`io._io.open`, `io.open_code` — the guard rebinds `io.open`, not the native reference it keeps as `io._io`). Every control here does what it was designed to do — COW-2433 deliberately placed the deny check ahead of the cache — and the capability survives the composition. That is the argument for the boundary in a single finding: these are residuals of closed tickets (COW-2284, COW-2418, both Done) that key off the deny list and miss `posix`; COW-2228, which would have closed the `builtins/exec` route, is Canceled.

Under the proposed boundary: the child profile denies process creation/network and exposes no useful host root or descriptors; a child panic dies at the process boundary. Parent-side parsing and commit still require their own panic-safe staged boundary.

8. Alternatives considered

Option	Finding	Decision
Continue Python-level blocklist/freeze hardening	Useful bridge control, but completeness cost recurs across names, types, VM fields, TLS, statics, and new RustPython versions	Keep only as defense in depth
Construct a minimal actor environment without a process boundary	Reduces native/ambient surface but does not isolate interpreter panic, memory-safety escape, or completeness errors	Required inside the child, insufficient alone
Fresh interpreter per actor	Simple isolation, measured 86-133 ms per frame	Not recommended at current block times
Fork once per transaction	Contains validator liveness but permits cross-actor reuse inside nested call trees	Does not meet the stated invariant
Restore a warm interpreter arena	Must prove VM/TLS/static completeness and risks unsafe restoration of Rust object graphs	Shelved; revisit only with a verified corpus and compelling throughput need
Sandboxed zygote + fork per actor frame	Discards private state, contains failures, and measured about 1.2 ms marginal cost	Recommended

9. Verification, provenance, and open questions

Source inspection used `cowboyinc/node` `origin/devnet` `cdb89a4e`, with #1106 on `eeeb0b5f` and #1110 on `caa201da`. PRs #1105, #1106, and #1110 were open and unmerged at the time of review.

Static claims carry source anchors inspected during the review. Spike measurements and the Trail of Bits draft were not independently revalidated in this document review. The CODE/BEEF `settrace` result depends on an uncommitted `execute_handler` harness. Spike sources (`zygote_spike.rs`, `pristine_probe.rs`, `prefault_spike.rs`, `checkpoint_probe.rs`) remain scratch artifacts.

Open validation items:

- production actor-frame count, nesting distribution, handler duration, reads, and effects;
- bare-metal x86-64 fork/CoW/IPC p50 and p99;
- the 86-133 ms cold-build figure on a live node, especially the first nested call path;

- complete read/write choke-point inventory;
- child and worker syscall traces from the real production path;
- deterministic zygote warm-up across supported validator builds;
- consensus policy for unexpected child infrastructure failure and replay;
- differential parity over production `execute_handler` and all nested host paths.

Appendix A - identified channel inventory

This is a floor, not a claim of completeness. It exists to seed the regression corpus; the architecture does not depend on finishing the list.

Channel	Class	Current mechanism
<code>rustpython_checkpoint.checkpoint*</code>	validator kill	process exit, tracked separately as COW-2731
<code>_signal.raise_signal(SIGKILL)</code>	validator kill	signal current validator process
<code>_posixsubprocess.fork_exec</code>	host escape	real fork/exec
<code>_socket.socket / connect</code>	host escape	real kernel network
<code>io.FileIO / _io.FileIO</code>	host escape	direct <code>crt_fd::open</code> path
native <code>_io.open</code> , <code>io.open_code</code>	host escape	cached import skips the allowlist; the guard rebound <code>io.open</code> , not native <code>_io.open</code>
<code>sys.settrace</code>	cross-actor execution	retained VM callback
<code>sys.setprofile</code>	cross-actor execution	retained VM callback
<code>threading.settrace</code>	cross-actor execution	<code>StoreGlobal</code> bypass into baseline module
<code>_thread.start_new_thread</code>	cross-actor mutation	spawned thread has barrier TLS off
<code>_thread.get_ident</code>	consensus input	raw process-local Rust thread id
<code>_thread._set_sentinel</code>	unbounded retained state	pooled thread sentinel list never clears
<code>sys.orig_argv</code>	consensus input	validator process arguments
<code>_socket.gethostname</code>	consensus input	validator host name
<code>_socket.setdefaulttimeout</code>	cross-execution state	process-global atomic
<code>sys.set_asyncgen_hooks</code>	cross-execution state	Rust TLS
<code>sys.set_coroutine_origin_tracking_depth</code>	cross-execution state	Rust TLS
<code>sys.set_int_max_str_digits</code>	cross-execution state	process-global atomic, no reset
<code>atexit.register</code>	cross-execution state	VM registry, no reset
<code>codecs.register / register_error</code>	cross-execution state	native codec registry
<code>copy.deepcopy</code> mutable default	cross-execution state	freeze walk stops at defaults tuple
closure-cell replacement	cross-execution mutation	closure cells not frozen/traversed
<code>_abc_registry WeakSet</code>	cross-execution state	container not frozen
property descriptors	cross-execution state	terminal nodes in freeze walk
lazy-import <code>sys.modules</code> persistence	cross-execution state	cache entry remains after execution
cold/warm <code>id()</code> sequence	consensus divergence	cold setup consumes ids that warm setup skips

The positive actor environment should also treat `_ssl` and `fcntl` as unassessed native capabilities until reviewed; they are not counted as confirmed live findings above.

Appendix B - low-level design obligations

- The zygote is single-threaded at every fork.

- No `MAP_SHARED` region carries actor-writable trusted state.
- The child receives one frame-specific endpoint and no ambient fds.
- Worker and child syscall profiles are generated from observed production traces and reviewed as allowlists.
- The child profile is installed before actor import/bytecode.
- Namespace-visible values are deterministic.
- Every IPC allocation is bounded before allocation.
- The parser rejects duplicates, unknown versions, trailing bytes, and non-canonical forms without panic.
- Parent reads and effects are capability-checked and transaction-bound.
- Nested frames fork from the zygote, never from an actor child.
- The overlay supplies read-your-writes and commits once.
- Wall-clock and resource kills are never committed transaction outcomes.
- The validator has an explicit replay/drop-out policy for infrastructure failures.
- The full production isolation corpus runs on every RustPython upgrade.